

```
gcc -o helloic -fopenmp hello1.c
```

This is to compile the code that we have written.

Next, specify to the OS to what extent we want to parallelise our code, i.e., how many threads we want, by using the following command:

```
export OMP_NUM_THREADS = 2
```

Once this is done, we can run our compiled code using the following command:

```
./filenamec
```

As you can see in Figure 1, it shows that we printed 'Hello' twice, once from each thread! That's cool, right?

If we have two tasks, both the tasks can run on separate threads parallelly using OpenMP. Even if we have a 'for' loop, each iteration of the loop runs on separate threads.

Advanced features of OpenMP

OpenMP has some advanced features for different use cases of parallel

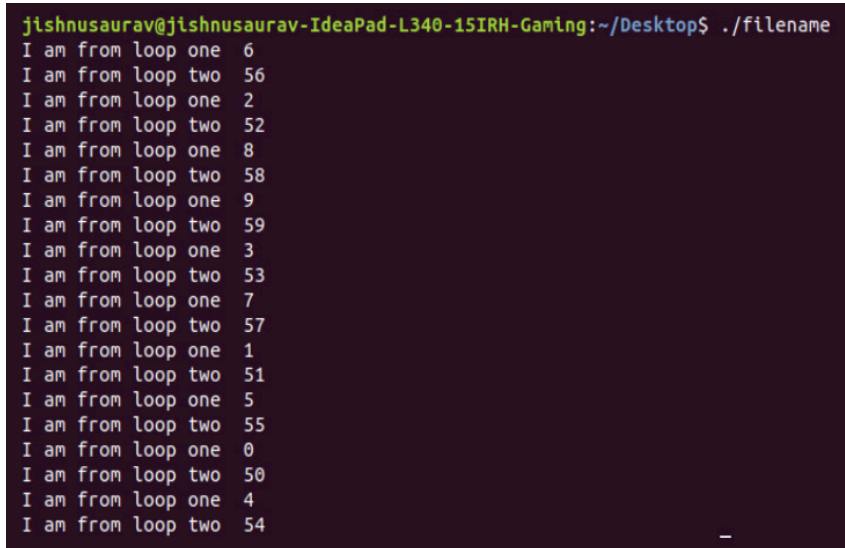
programming. The first feature that we are going to discuss here is the 'nowait' feature. If we have two commands/'for' loops that we want to run parallelly and don't want that one 'for' loop waits for the other to complete, i.e., if we want to run all the iterations of both loops parallelly, this feature is used. You can test this using the following piece of code and the output can be seen in Figure 2.

```
#include <omp.h>
#define _GNU_SOURCE
#include <stdio.h>
#include <sched.h>
int main()
{
    int N; N=10;
    #pragma omp parallel
    {
        #pragma omp for nowait
        for (int i=0; i<N; i++)
            printf("I am from loop one %d\n",i);
        #pragma omp for
        for (int i=0; i<N; i++)
            printf("I am from loop two %d\n",i+50);
    } }
```

As we see in Figure 2, since we have used 'nowait', loop 2 does not wait until loop 1 is over.

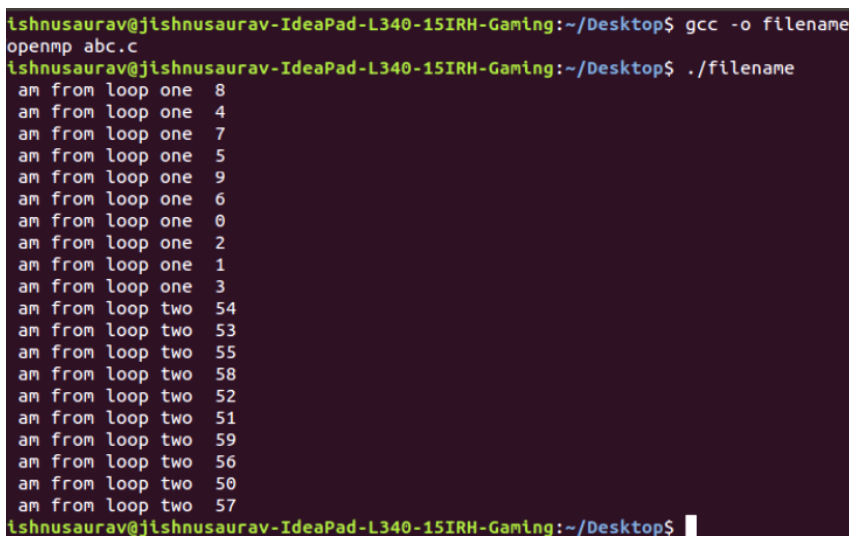
The next interesting feature that OpenMP has is 'barrier'. Using this feature, we can ask OpenMP to wait for the first loop to complete running parallelly before moving to the next loop. In this the iterations of the first and second loops run parallelly but the loops themselves don't run parallelly. This is especially useful in situations when we want the output from the first loop to be used in the second. You can test this feature using the following code and the output is shown in Figure 3.

```
#include <omp.h>
#define _GNU_SOURCE
#include <stdio.h>
#include <sched.h>
int main()
{
```



```
jishnusaaurav@jishnusaaurav-IdeaPad-L340-15IRH-Gaming:~/Desktop$ ./filename
I am from loop one 6
I am from loop two 56
I am from loop one 2
I am from loop two 52
I am from loop one 8
I am from loop two 58
I am from loop one 9
I am from loop two 59
I am from loop one 3
I am from loop two 53
I am from loop one 7
I am from loop two 57
I am from loop one 1
I am from loop two 51
I am from loop one 5
I am from loop two 55
I am from loop one 0
I am from loop two 50
I am from loop one 4
I am from loop two 54
```

Figure 2: 'Nowait' output



```
jishnusaaurav@jishnusaaurav-IdeaPad-L340-15IRH-Gaming:~/Desktop$ gcc -o filename openmp abc.c
jishnusaaurav@jishnusaaurav-IdeaPad-L340-15IRH-Gaming:~/Desktop$ ./filename
am from loop one 8
am from loop one 4
am from loop one 7
am from loop one 5
am from loop one 9
am from loop one 6
am from loop one 0
am from loop one 2
am from loop one 1
am from loop one 3
am from loop two 54
am from loop two 53
am from loop two 55
am from loop two 58
am from loop two 52
am from loop two 51
am from loop two 59
am from loop two 56
am from loop two 50
am from loop two 57
jishnusaaurav@jishnusaaurav-IdeaPad-L340-15IRH-Gaming:~/Desktop$
```

Figure 3: 'Barrier' output